



École Polytechnique Fédérale de Lausanne

Reliability-Aware Routing In Distributed Surface-Code
Quantum Architectures

by David Gorgiev

Bachelor Project Report

Approved by the Examining Committee:

Prof. Edoardo Charbon
Thesis Advisor

Batuhan Keskin
Thesis Supervisor

EPFL
Advanced Quantum Architecture Laboratory
CH-1015 Lausanne

June 6, 2026

Abstract

Distributed surface-code architectures are among the most promising architectures for large-scale fault-tolerant quantum computers [4, 15, 19]. In such architectures, routing logical states through ancilla patches becomes a central compilation and runtime-control problem.

This project focuses on the fact that the shortest route is not necessarily the most reliable route. Existing routing approaches typically optimize geometric distance or congestion [10, 15, 17], but they do not use the syndrome information continuously produced by the surface code to estimate fidelity risk and avoid temporarily noisy regions of the processor [3, 9].

We propose a reliability-aware runtime architecture that turns syndrome activity into a reliability map over logical patches. The controller uses this map during routing, gate compilation, and virtual-qubit placement, so that logical states can be moved through safer patches or placed directly on safe interaction patterns.

The implementation is a discrete-event simulator for distributed rhombic surface-code layouts, following the work of Lovis [16]. It decomposes logical gates into atomic operations, tracks resource reservations, supports fixed-data and virtual-qubit strategies, and compares shortest-path and reliability-aware control policies.

The evaluation shows that reliability awareness can substantially improve simulated success probability. In a single-core hotspot experiment, fixed reliability-aware routing improves true success probability from 0.435 to 0.953 with a two-cycle latency cost, while reliability-aware virtual mapping reaches 0.982 without extra latency. In a distributed noisy-core experiment, reliability-aware virtual mapping improves true success probability from 0.502 for reliability-blind virtual mapping to 0.933 while keeping the same 18-cycle latency.

Contents

Abstract	2
1 Introduction	5
2 Background	7
2.1 Rhombic layout and Virtual qubits	7
2.2 Distributed cores	7
2.3 Routing	8
2.4 Error detection	8
3 Design	9
3.1 Ancilla network as a weighted graph	9
3.2 Reliability map	10
3.2.1 Error rate estimate	10
3.2.2 Usage bias correction	11
3.3 Routing policy	12
3.3.1 Stalling and Scheduling	12
3.3.2 Routing	13
3.4 Gain-based dynamic rerouting threshold	14
4 Implementation	16
4.1 Simulator overview	16
4.2 Config Loader	18
4.3 Experiment Runner	19
4.4 Runtime Controller	20
4.5 Template generator	21
4.6 Compilation, Reservation Table, Qubit Mapping	23
4.6.1 Time-slice scheduler	23
4.6.2 Qubit mapping	23
4.6.3 Reservation table	24
4.7 Label-setting Routing algorithm	24
4.8 Reliability map	25

4.9	Noise model	25
4.10	Strategies	26
5	Evaluation	28
5.1	Experiment 1: single CNOT in a single core	28
5.1.1	Route choices	31
5.2	Experiment 2 : distant CNOTs in distributed cores	38
5.2.1	Route choices	40
6	Future Work	49
6.1	Circuit-Level Noise Modeling with STIM	49
6.2	Reliability-Architecture Ablation and Parameter Optimization	50
6.3	Stricter Rhombic Core Layout	50
6.4	Scaling to Larger Circuits and Architectures	50
7	Conclusion	52
	Bibliography	54

Chapter 1

Introduction

Quantum computing platforms are evolving to a point where placing high numbers of qubits into a single core comes with certain difficulties such as fidelity, crosstalk, and high power consumption of dense classical electronics, to name a few. Having distributed cores, each hosting logical data qubits and logical ancillas connected via classical and quantum communication channels offers a promising alternative [4, 7, 15, 19]. However, building such a system for logical qubits instead of physical qubits needs additional optimizations, such as minimizing the number of state transfer between cores for inter-core two-qubit gates and optimizing the routing of magic states distilled in a magic state factory [5, 14, 15].

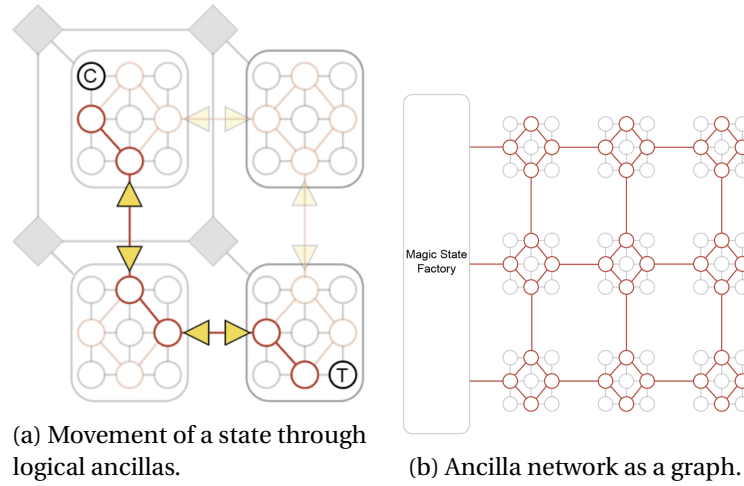


Figure 1.1: Routing and graph representation.

Surface code only supports nearest-neighbor operations [3, 12]. Thus, to implement a two-qubit gate between distant qubits in a distributed surface code layout, we need to move a logical data qubit next to the other one either by traversing logical ancilla patches, or by teleporting it [19, 20].

Moving the qubit through the ancillas requires many lattice surgery operations, which leads to higher accumulated logical error probability [2, 12]. Therefore, the routing of qubits in existing architectures is typically computed as the shortest path between the two qubits [10, 15, 17]. However, certain qubits can become more or less noisy than others during execution [13, 18]. Thus, the shortest path may not always be the safest one.

The idea of this work is to improve the overall fidelity of quantum circuit execution by introducing feedback from the periodic stabilizer measurements to the routing and scheduling layer in order to dynamically reroute qubits at runtime and avoid noisy regions. Additionally, this architecture allows us to choose reliable mappings of virtual-qubits to logical-qubit patches. The contributions of this work are as follows:

- We propose a reliability-aware routing architecture and algorithms that leverage stabilizer measurement outcomes to construct a runtime reliability map and dynamically reroute and map qubits away from error-prone regions.
- We evaluate the proposed approach in terms of fidelity improvement, execution time, and space-time overhead, and compare it against the standard shortest-path routing strategy. We simulate both fixed- and virtual-qubit architectures in a rhombic layout. For this purpose, we implemented a discrete-event based simulator in python, which we provide a detailed description of.

Chapter 2

Background

In this chapter, we state and describe the assumptions about the architecture and constraints.

2.1 Rhombic layout and Virtual qubits

Although our reliability-aware architecture is independent of the surface-code geometry, our simulation uses a rhombic layout. The simulation also compares fixed-qubit architectures and virtual-qubit mapping. For both rhombic layout and virtual qubits, we refer to the original work of Lovis, which this work is a continuation of [16].

2.2 Distributed cores

To protect qubits from errors, logical qubits are encoded into rotated surface code patches of arbitrary size [3, 9]. There are logical data qubit patches holding the actual quantum information of interest, and logical ancilla patches needed to implement certain operations on logical data qubits, or to transfer logical states from one area of the Quantum Processing Unit (QPU) to another [2, 12].

Logical data qubits and logical ancillas are grouped into cores of arbitrary size, such that for any two logical data qubits inside a core, there is a path of logical ancillas connecting them. The QPU is composed of an arbitrary number of cores, and every core is connected to their nearest neighbors through a quantum teleportation channel involving a logical ancilla from each core [4, 7, 15, 19].

By essence, surface code restricts multiple-qubit operations to nearest-neighbor operations [3]. Therefore, to implement two-qubit gates, we need to move a logical data qubit's state next to the second logical data qubit. To achieve this, the state is transferred across a path of ancillas. Intra-core

state transfers are done using lattice surgery merge and split operations [2, 12]. Inter-core transfers are done by teleporting the state from a boundary logical ancilla to another [19, 20].

2.3 Routing

We refer to routing as the process of computing a route that connects two logical data qubits through a path of logical ancillas. Routing constitutes an important step in the quantum computing pipeline with surface code cores, as routing decisions can significantly impact state fidelity. Intuitively, a state transfer requiring going through an infinite amount of ancillas would probably accumulate an infinite amount of errors. In comparison, a state transfer needing only one intermediate ancilla would have a negligible error probability.

Therefore, the routing is typically computed as the shortest path of ancillas between the two logical data qubits [10, 15, 17]. However, in practice, the error rate is not uniform over every qubit of the QPU [13, 18]. Hence, the shortest path may not be the most reliable if it goes through noisy patches. A longer path that uses less noisy ancillas can theoretically have a smaller error probability. We formalize this problem in chapter 3.

2.4 Error detection

The standard method to detect errors in surface code patches is through stabilizer measurements and syndrome extraction [3, 9]. Independently and in parallel of the circuit running, a Syndrome Extraction Unit periodically performs stabilizer measurements over every patch of the QPU and computes the corresponding syndromes. We will use this information to track and assign error rates to each logical ancilla at runtime, and dynamically reroute state transfers.

Chapter 3

Design

The routing problem can be mapped to a path-finding problem in a weighted graph of ancillas. In this chapter, we formalize the problem, describe the weight assignment algorithm, and introduce a routing policy.

3.1 Ancilla network as a weighted graph

Let $G = (V, E, W)$ a weighted undirected graph representing the ancilla network, with V the set of nodes, E the set of edges, W the set of weights, defined as follows.

- Each ancilla is represented by exactly one node $q_i \in V$.
- $(u, v) \in E \iff u \neq v$ and u, v can directly interact by neighbor lattice surgery operations or by inter-core teleportation.
- Every $q_i \in V$ has exactly one associated weight $w(q_i) \in W$ modeling the probability $p(q_i) = w(q_i)$ of a Pauli error occurring in ancilla q_i during a state transfer.

Let the ancilla q_A contain the logical state $|\Psi\rangle$ to be transferred to q_B . We define the path from node q_A to node q_B as $C(q_A, q_B) = \{q_1, q_2, \dots, q_n\}$, with $q_1, \dots, q_n$ the intermediate nodes taken along the path. The probability $P_C(|\Psi\rangle)$ of $|\Psi\rangle$ suffering from a Pauli error along the path C is

$$P_C(|\Psi\rangle) = 1 - \prod_{q_i \in C} (1 - p(q_i)) \quad (3.1)$$

. For two paths $C_i(q_A, q_B), C_j(q_A, q_B)$ that have the same starting node and target, we define the fidelity gain $G(C_i, C_j)$ of the path C_i over the path C_j as the difference between their cumulative

error probabilities

$$G(C_i, C_j) = P_{C_i}(|\Psi\rangle) - P_{C_j}(|\Psi\rangle) \quad (3.2)$$

3.2 Reliability map

3.2.1 Error rate estimate

In section 3.1, we have defined the weight of a node as the error rate of the associated ancilla during a state transfer. This value is assumed to be unknown a priori. Therefore, we approximate it online from the outcomes of the stabilizer measurements produced by the Syndrome Extraction Unit, which periodically monitors all logical patches of the QPU in parallel of the execution.

For each logical ancilla patch q_i , let $s_i^{(t)}$ be the syndrome bitstring observed at quantum error correction round t . We consider the number of detection events, i.e. the number of stabilizer outcomes that changed between two consecutive rounds. For two syndrome bitstrings $s_i^{(t-1)}$, $s_i^{(t)}$ measured respectively at round $t-1$ and t , and associated to ancilla q_i , the detection event count $d_i^{(t)}$ is

$$d_i^{(t)} = \text{popcount}(s_i^{(t-1)} \oplus s_i^{(t)}) \quad (3.3)$$

, where $\oplus$ is the bitwise XOR, and popcount returns the number of bits set to 1. Intuitively, a high number of detection events in a patch means that it is currently experiencing more error activity and should therefore be considered less reliable.

We assume that the number of stabilizer is fixed for each patch, but can vary between patches. Therefore, we should normalize the detection event count to obtain a corrected per-round error activity indicator $r_i^{(t)} \in [0, 1]$:

$$r_i^{(t)} = \frac{d_i^{(t)}}{N_{stab,i}} \quad (3.4)$$

. To avoid overreacting to transient fluctuations, we do not use this value directly as our weights. Instead, we integrate it into an exponentially weighted moving average $\rho_i^{(t)}$ with

$$\rho_i^{(t)} = (1 - \alpha)\rho_i^{(t-1)} + \alpha r_i^{(t)}, \quad (3.5)$$

where α is a tunable parameter.

3.2.2 Usage bias correction

The raw syndrome activity of an ancilla can be biased by how often the ancilla is used. A frequently used ancilla is involved in more state-transfer operations and may therefore produce more detection events than an idle ancilla, even if it is not intrinsically less reliable. Directly penalizing such ancillas would bias the router against central routing resources and could produce unnecessarily long routes.

To reduce this usage bias, we normalize the reliability estimate by the recent usage of each ancilla. Let $m_i^{(t)}$ denote the number (0 or 1 usually) of operations involving ancilla q_i during runtime cycle t . We define a smoothed usage estimate

$$u_i^{(t)} = (1 - \gamma)u_i^{(t-1)} + \gamma m_i^{(t)}, \quad (3.6)$$

where γ controls how quickly past usage is forgotten.

Let $\rho_i^{(t)}$ denote the smoothed raw syndrome activity of ancilla q_i . We define the usage-corrected reliability estimate as

$$\tilde{\rho}_i^{(t)} = \frac{\rho_i^{(t)}}{1 + u_i^{(t)}}. \quad (3.7)$$

The denominator represents the recent exposure of the ancilla. The constant 1 avoids division by zero and represents the baseline exposure due to periodic stabilizer measurements, which occur even when the ancilla is idle. Therefore, an ancilla is considered risky only if it exhibits high syndrome activity relative to how much it has recently been used.

We can now construct the reliability map by initializing all weights as zero-risk ($\rho_i^{(0)} = 0$ and $u_i^{(t)} = 0, \forall i$) and updating them at each round such that

$$w_t(q_i) = \tilde{\rho}_i^{(t)}. \quad (3.8)$$

This gives the routing layer access to a view of the spatial distribution of error activity across the QPU at runtime.

3.3 Routing policy

The routing layer is responsible for deciding when a planned state transfer route should be recomputed using updated reliability information. Since the reliability map evolves during execution, as stabilizer checks are measured, a route that was considered safe at scheduling time may become unreliable later. However, recomputing multiple routes after every update would introduce excessive runtime overhead. Therefore, rerouting is triggered only when the reliability of a planned route degrades beyond an admissible limit.

3.3.1 Stalling and Scheduling

For every planned route C , the controller maintains an estimate of its cumulative error probability at cycle t as

$$P_t(C) = 1 - \prod_{q_i \in C} (1 - w_t(q_i)) = 1 - \prod_{q_i \in C} (1 - \tilde{\rho}_i^{(t)}) \quad (3.9)$$

This value is updated at every runtime cycle for all active or imminent planned routes. A route C is considered unsafe when

$$P_t(C) > \tau_{\text{reroute}}^{(t)}, \quad (3.10)$$

where $\tau_{\text{reroute}}^{(t)}$ is the current rerouting threshold.

If this condition is met, the route is flagged and all subsequent scheduled operations are stalled until rerouting and rescheduling is done.

To avoid conflicts between dynamically rerouted transfers and already scheduled operations, we adopt a conservative global rerouting barrier. When a planned route exceeds the rerouting threshold, the scheduler stops issuing new operations that require logical ancillas. Operations that are already active are allowed to complete, after which the ancilla network is considered idle. The routing layer then recomputes a replacement route using the current reliability map on the full weighted graph. Since no other operation is scheduled during this rerouting window, the router does not need to reason about time-dependent ancilla availability. After the rerouted transfer has been scheduled and executed, normal scheduling resumes.

This design significantly simplifies the rerouting problem by avoiding parallel resource conflicts. The cost is additional execution latency and reduced parallelism, since independent operations may also be stalled during rerouting. Therefore, this policy is conservative but suitable as a first implementation of reliability-aware dynamic rerouting.

3.3.2 Routing

The routing layer computes a candidate replacement route C' between the same source and destination ancillas. The objective is not to find the globally minimum-risk path, since such a path could be unnecessarily long and could negatively affect latency and parallel execution. Instead, the router searches for the shortest route satisfying

$$P_t(C') < \tau_{\text{safe}}, \quad (3.11)$$

where τ_{safe} is the minimal safety constraint imposed on every route. It makes sense to choose $\tau_{\text{safe}} \leq \tau_{\text{reroute}}$.

The rerouting problem is thus formulated as a Resource-Constrained Shortest-Path problem with hop count as the primary optimization objective, and estimated cumulative route error probability as the constraint [1].

Equivalently, the selected route is

$$C' = \underset{C}{\operatorname{argmin}} |C| \quad (3.12)$$

subject to

$$1 - \prod_{q_i \in C} (1 - \tilde{\rho}_i^{(t)}) \leq \tau_{\text{safe}}. \quad (3.13)$$

The router solves this problem using a label-setting resource-constrained shortest-path algorithm [1]. Each partial route is represented by a label containing the current node, the number of hops, the accumulated risk, and a predecessor pointer for reconstructing the path. Labels whose accumulated risk already exceeds τ_{safe} are discarded. Labels that are worse than another label reaching the same node are also discarded by dominance pruning. A label ℓ_1 dominates another label ℓ_2 at the same node if

$$h(\ell_1) \leq h(\ell_2) \quad (3.14)$$

and

$$P(\ell_1) \leq P(\ell_2), \quad (3.15)$$

with at least one strict inequality, where $h(\ell)$ is the hop count and $P(\ell)$ is the accumulated risk of

label ℓ .

When a feasible route reaches the destination, the router returns the feasible route with the smallest hop count. If no feasible route exists under the current safety threshold, the scheduler keeps the operation stalled and retries after the reliability map is updated. To avoid stalling infinitely in the case where there would be no valid path for a long time, we set a maximum stalling window after which we would run the routing algorithm with $\tau_{\text{safe}} = 1$, i.e. taking the safest possible route.

3.4 Gain-based dynamic rerouting threshold

A fixed rerouting threshold can cause two undesirable behaviors. If the threshold is too low, the system may reroute too often and increase execution time. If the threshold is too high, the system may continue using unreliable routes even when a better alternative exists. Therefore, to adapt the rerouting policy during execution, we update the rerouting threshold based on the estimated gain obtained after each rerouting event.

When a planned route C exceeds the current rerouting threshold, rerouting is triggered:

$$P_t(C) > \tau_{\text{reroute}}^{(t)}. \quad (3.16)$$

The scheduler then stalls execution and computes a new route C' using the current reliability map. If a feasible route is found, the original route is replaced by the new route. The controller then estimates the gain of the rerouting decision as

$$G_t(C, C') = P_t(C) - P_t(C'). \quad (3.17)$$

A positive value means that the new route has a lower estimated error probability than the original route. This gain is then used as feedback to tune the rerouting threshold for future decisions:

$$\tau_{\text{reroute}}^{(t+1)} = \text{clip}\left(\tau_{\text{reroute}}^{(t)} + \beta(G_{\text{target}} - G_t(C, C')), \tau_{\min}, \tau_{\max}\right). \quad (3.18)$$

Here, G_{target} is the desired gain from a rerouting event, β controls the adaptation speed, and $\tau_{\min}$ and $\tau_{\max}$ bound the threshold. Therefore, if rerouting produces a large gain, then $G_t(C, C') > G_{\text{target}}$, and the threshold is decreased. This makes future rerouting more aggressive, because previous

rerouting decisions were beneficial. If rerouting produces only a small gain, then $G_t(C, C') < G_{\text{target}}$, and the threshold is increased. This makes future rerouting less aggressive, because previous rerouting decisions gave little improvement.

Furthermore, it makes sense to maintain the safety constraint τ_{safe} under the rerouting threshold τ_{reroute} to guarantee that the new route is safer than the former. We can therefore set

$$\tau_{\text{safe}}^{(t)} = \lambda \tau_{\text{reroute}}^{(t)} \quad (3.19)$$

with $0 < \lambda \leq 1$.

Chapter 4

Implementation

The implementation is a discrete-event simulator for evaluating the routing architecture described in chapter 3. It does not simulate the full quantum state vector of the input circuit. Instead, it simulates the control decisions that determine where logical states move, which physical resources are reserved, how long execution takes, and which logical error probabilities are accumulated along the way. This makes the simulator suitable for comparing routing policies over the same layout, circuit, and noise model.

The code is written in Python. The simulator takes as input an experiment configuration containing a rhombic layout, a logical circuit, timing parameters, a hidden noise model, and a set of strategies to evaluate. For each strategy and repetition, the simulator executes the circuit in discrete surface-code cycles and records events, route choices, reliability-map snapshots, and aggregate metrics. Finally, it outputs execution traces, metrics, and figures.

4.1 Simulator overview

Figure 4.1 shows a high-level representation of the program flow. The command-line entry point reads an experiment file, builds the corresponding data model, and dispatches one run per configured strategy and repetition. Each run constructs the same logical layout and circuit, but uses a different routing and compilation strategies.

We can distinguish the following components in the simulator:

- **Configuration loader:** parses the JSON experiment file and validates the layout, circuit, timing, algorithm, and noise parameters.
- **Layout and circuit model:** stores surface-code patches, edges, legal measurements, logical

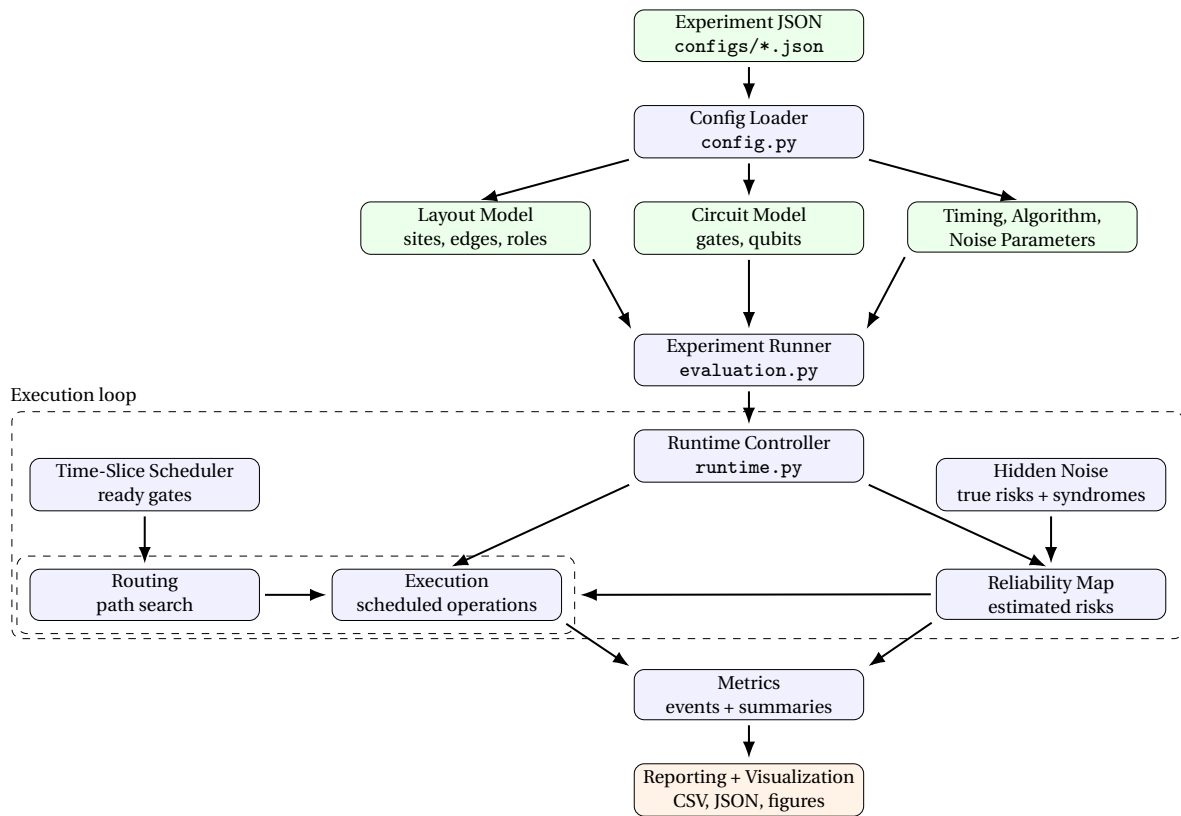


Figure 4.1: Component interaction graph of the current simulator.

gates, and logical qubit mappings.

- **Experiment runner:** iterates over strategies and repetitions, constructs a runtime instance for each run, and writes the resulting reports.
- **Runtime controller:** advances the simulation clock, completes scheduled operations, updates the reliability map, checks rerouting conditions, and schedules ready gates.
- **Template generator:** expands a logical gate into one or more possible physical implementations using atomic operations such as joint measurements, magic state imports, or frame update.
- **Router:** searches paths through the layout graph under timing, reservation, and reliability constraints.
- **Reservation table:** prevents simultaneous use of the same site or edge by overlapping operations.
- **Hidden noise model:** generates noise in patches, edges, joint-measurements, magic-states, idle states, and syndrome detection.
- **Reliability map:** estimates per-patch risks at runtime from syndrome samples and supplies those estimates to the router and scheduler.
- **Metrics and reporting:** stores event traces, reliability traces, mappings, summary rows, plots, and Markdown/HTML reports.

4.2 Config Loader

All parameters are provided through JSON configuration files. The loader is implemented in `config.py`. It reads the JSON file, constructs dataclass instances for the timing, algorithm, and noise parameters, and converts the layout and circuit dictionaries into typed objects.

Each experiment file contains:

- The hardware layout: cores, patches (type and position), stabilizer counts, base patch errors, neighbor edges, legal joint measurements, and inter-core communication links.
- The logical circuit to execute.
- The initial fixed mapping from logical qubits to logical data patches for fixed-qubits strategies.
- The list of strategies to run, the number of repetitions, the random seed, and the output directory.

- Timing parameters such as syndrome period, smooth teleportation duration, inter-core teleportation duration, joint-measurement duration, magic state import duration, wait duration, scheduling epoch, and maximum cycle budget.
- Algorithm parameters such as the reliability-map smoothing factors, rerouting threshold, safety factor, adaptive-threshold parameters, routing horizon (how far in the future can the router search for a feasible route), and batch size (maximum number of gates the scheduler is allowed to schedule per cycle).
- Noise parameters such as patch error, measurement error, usage boost, sinusoidal drift amplitude, drift period, and space-time hotspots.

The loader validates several consistency conditions before an experiment starts. The layout graph must be connected, every edge endpoint must refer to an existing patch, every edge measurement must be one of the supported measurements MXX , MZZ , MXZ , or MZX , and every fixed logical mapping must refer to a distinct data patch. For fixed-qubit strategies, the number of data patches must be at least the number of logical qubits in the circuit.

To simplify experiment construction, the project also includes a local visual config editor in `config_editor.py`. The editor serves a small web interface, lets the user place patches and edges visually, edit circuit gates and parameters, validate the resulting experiment, and export the config as a JSON file into the `configs` directory.

4.3 Experiment Runner

The experiment runner is implemented in `evaluation.py`. After the configuration has been loaded, the runner creates the output directory and runs the simulation with every specified strategy. For every repetition r , a new seed is generated. The same repetition seed is used across all strategies, which makes the hidden noise process comparable across strategies within a repetition.

When the circuit execution finishes, the runner collects three main artifacts:

- A summary row containing latency, reliability, routing, and rerouting metrics.
- An event trace containing timestamped operation completions, gate completions, syndrome updates, reroutes, fallbacks, and failures.
- A reliability trace containing site and edge risk estimates at every syndrome-update cycle.

After all runs are complete, the experiment runner calls the reporting layer to write raw CSV/JSON traces, derived metric tables, plots, route-choice plots, and a compiled report.

Algorithm 1 Runtime controller loop

```
1: build hidden noise model, reliability map, router, templates, reservations, and compiler state
2: while the schedule is not complete do
3:   if current time exceeds max_cycles then
4:     mark the run as failed and stop
5:   end if
6:   complete all atomic operations whose end cycle has arrived
7:   if all logical gates are complete then
8:     stop successfully
9:   end if
10:  if the current cycle is a syndrome-update cycle then
11:    sample hidden syndrome observations and update the reliability map
12:  end if
13:  if the selected strategy supports partial rerouting then
14:    inspect current route suffixes and reroute unsafe ones
15:  end if
16:  select ready logical gates, schedule and route them
17:  advance time by one scheduling epoch
18: end while
```

4.4 Runtime Controller

The runtime controller, implemented in `runtime.py`, contains the main discrete-event simulation loop. Time is represented as an integer number of surface-code cycles. The simulator advances by `scheduling_epoch` cycles, which is one cycle in the current configurations, and stops when all gates in the schedule are completed or when `max_cycles` is exceeded.

Every logical gate and routing movement is decomposed by the template generator into the following atomic operations: RESET, MEASURE, JOINT_MEASURE, and MAGIC_STATE_IMPORT. Each operation stores its start and end cycles, the logical gate it belongs to, the logical qubits it touches, the resources it reserves, its estimated error, its hidden true error, and operation-specific metadata. When an operation completes, the runtime multiplies the run's true success probability by $1 - p_{\text{true}}$ and the estimated success probability by $1 - p_{\text{est}}$.

For fixed-qubits strategies, the logical data patches remain the persistent homes of the logical qubits. Movement is treated as a temporary operation used to implement an operation, and only the tracked Pauli and Hadamard frame are kept after the gate. For virtual qubit strategies, their logical patch changes persist until the qubit is deactivated after its final circuit use.

Dynamic routing strategies do partial rerouting checks : even once a logical gate has been routed and is currently being executed, if the remaining route to travel becomes unsafe, the execution of the gate is stopped to find a new safe path for the remaining operations. More precisely, the

implementation does not interrupt an atomic operation that is already active. It inspects scheduled templates of execution whose future operations have not started yet. For such a plan, the runtime recomputes the estimated risk of the remaining suffix using the current reliability map. If that suffix risk exceeds the current rerouting threshold τ_{reroute} , the runtime releases the future reservations of the plan, marks the plan as canceled, pushes the logical gate in the time slice's pending gate set, and lets the scheduler generate a replacement plan.

For strategies using the adaptive rerouting threshold, if the estimated risk of a plan changes from its original estimate P_{old} to a current suffix estimate P_{new} , the threshold is updated as

$$\tau_{\text{reroute}} \leftarrow \text{clip}_{[\tau_{\min}, \tau_{\max}]} (\tau_{\text{reroute}} + \beta (G_{\text{target}} - (P_{\text{old}} - P_{\text{new}}))). \quad (4.1)$$

The safety threshold used by reliability-aware route search is always derived from the current rerouting threshold as

$$\tau_{\text{safe}} = \lambda \tau_{\text{reroute}}. \quad (4.2)$$

4.5 Template generator

The template generator, implemented in `templates.py`, translates ready logical gates into physical execution plans. A `TemplatePlan` is a sequence of atomic operations, estimated and true error probabilities, a cost, final logical-qubit states, the number of route hops, and a space-time volume (number of cycles reserved * number of patches reserved). To schedule a logical gate, the runtime controller asks the template generator for candidate execution plans and then chooses one according to the active strategy.

The supported gate classes are:

- **Frame gates:** H , X , Y , and Z are represented as frame updates. They emit an IDLE atomic operation with `operation_role = frame_update`. They reserve no resources and have `duration timing.frame_cycles`, defaulting to 0.
- **Magic gates:** T , $T^\dagger$, S , $S^\dagger$, and R_Z are implemented as Lovis fig. 1 circuits [16]. The generator routes the logical data qubit, when needed, to a patch adjacent to a magic patch. It then imports the magic state at the magic patch, performs an MZZ joint measurement between the data and magic patches, and measures the consumed magic patch in the MX basis. Since the simulator is discrete event-based, and does not concretely perform the computations and measurements, the corrections controlled on the MX outcome are recorded as frame corrections for simplicity.
- **Two-qubit gates:** CX , $CNOT$, and CZ are implemented as Lovis fig. 9 two-qubit circuits family [16]. The generator chooses a shared ancilla patch adjacent to the two operand patches,

routing one or both operands when necessary. The circuit then resets the ancilla, performs two joint measurements, and finally measures either the ancilla or, for the moving-control/moving-target variants (Lovis fig. 9(d)), the source operand patch. Similarly, the controlled corrections are recorded as frame updates for simplicity. Related error channels for lattice-surgery CNOTs between surface-code patches are characterized in [6].

For a two-qubit gate, the generator enumerates four families of plans. First, it checks whether the two operands are already adjacent and whether the edge supports a legal joint measurement; if so, it creates a `direct` plan. Second, it tries `move_control`, where the control qubit is routed to a patch adjacent to the target. Third, it tries `move_target`. Fourth, it tries `move_both`, where both operands are routed to the endpoints of a legal edge. The `move_both` enumeration can create many candidates, so the implementation filters dominated plans and keeps the first twelve nondominated candidates, ordered by estimated error, end time, and hop count.

Movement operations are generated from the route returned by the router. Each intra-core route hop is expanded into the Lovis fig. 8 smooth teleportation circuit: a RESET on the destination patch, a JOINT_MEASURE across the source-destination edge, and a MEASURE of the old source patch [16]. Inter-core teleportations are expanded into Keskin et al. fig.1 teleportation circuit: the transmitting and receiving communication patches are reset as EPR halves, an EPR pair is generated across the inter-core link, a CNOT is applied between the data patch and the transmitting EPR half, the source-side patches are measured, classical communication is represented by an IDLE operation, and X/Z frame corrections are recorded on the receiving communication patch [15].

The generator tracks frame information in the virtual qubit state. Smooth-teleportation measurement choice depends on the current Hadamard frame, and frame-flipping edges update that frame. For the two-qubit templates, feasibility is determined by shared ancilla geometry and supported edge measurements. The effective gate under tracked frames is recorded in operation metadata.

For reliability-aware strategies, template choice is based on a weighted cost:

$$\text{cost} = a_r \hat{p} + a_s V + a_m M + a_l L, \quad (4.3)$$

where $\hat{p}$ is the estimated plan error, V is site space-time volume, M is a magic-import indicator, and L is the number of route hops. The coefficients are configurable through the config file. Shortest-path strategies instead choose primarily by hop count, then end time, estimated error, and cost.

4.6 Compilation, Reservation Table, Qubit Mapping

4.6.1 Time-slice scheduler

The scheduler implement the time-slicing algorithm from Lovis 4.3.1, implemented in `lovis_compiler.py` [16]. It greedily packs the input circuit into conflict-free time slices such that no two gates in the same slice touch the same logical qubit.

For each time slice, it tracks pending gates, issued gates, and completed gates. During each scheduling epoch, the runtime tries to schedule every pending gate in the current slice, and every gate whose resources are free can execute in parallel.

If a pending gate cannot find a feasible plan, or if all candidate plans conflict with already reserved resources, it remains in the current slice's pending set. The runtime retries pending gates on later scheduling epochs. The scheduler advances to the next time slice only when every gate in the current slice has completed.

4.6.2 Qubit mapping

In fixed-qubit strategies, the layout must contain enough logical data patches for the logical qubits in the input circuit, and the config file must provide a one-to-one initial mapping. Magic and communication patches are treated as ancilla patches with additional capabilities. Magic patches are possible entry points of magic states provided by an external magic state factory. Communication patches are situated at end points of inter-core links, and are the entry points of EPR pair halves provided by an external EPR pair generator for inter-core quantum teleportations. The number, type and placement of each patch must be provided by the input config file.

For virtual-qubit strategies, the input mapping of logical qubits to data patches is ignored. There is also no distinction between data and ancilla patches. However, the magic and communication patches remain. A logical circuit qubit is mapped on the physical layout when a gate in the current time slice needs it and that it is not already mapped. For a two-qubit gate with both operands missing, the mapper first tries to place the operands directly on free patches which share a free ancilla so that the gate can be executed directly without routing. If no such placement is available in the current cycle, it chooses a nearby placement from which routing can reach a legal shared ancilla. If only one operand is missing, the same logic is applied relative to the already active partner [11, 16].

For reliability-aware virtual-qubit strategies, virtual-qubit mapping prefers patches with lower estimated risk from the reliability map.

4.6.3 Reservation table

The reservation table in `reservation.py` records the cycle intervals during which each patch or edge is reserved for an operation. Each reservation is represented by a resource type, a resource id, a start cycle, and an end cycle. Before a plan is dispatched, the runtime controller asks the reservation table whether every resource interval in the plan is conflict-free against existing reservations. If so, the table reserves the full footprint of the plan. If not, the plan cannot be dispatched in the current scheduling epoch, therefore the gate is pushed back in the current time slice's pending gate set.

For reliability-aware strategies, at syndrome-update time, the table reports which sites are actively reserved, allowing the hidden noise model and reliability map to account for active usage. Therefore frequently used ancillas will be noisier. For strategies allowing partial rerouting, the table releases future reservations belonging to a canceled plan while preserving reservations for operations that already started.

4.7 Label-setting Routing algorithm

The router is implemented by `LabelSettingRouter` in `router.py`. The layout is treated as an undirected graph whose vertices are logical patches and whose edges are legal smooth-teleportation or inter-core communication links. The router receives a source site, a set of acceptable target sites, a start time, the current Hadamard frame of the moving logical qubit, the reliability map, the reservation table, a safety threshold, and a route mode [1].

Each partial path is represented by a label

$$\ell = (q, t, h, \hat{p}, p_{\text{true}}, f, C, M, R), \quad (4.4)$$

where q is the current site, t is the current end time, h is the hop count, $\hat{p}$ is the accumulated estimated risk, p_{true} is the accumulated hidden true risk used only for the metrics report, f is the Hadamard-frame bit, C is the path, M is the sequence of selected movement measurements, and R is the accumulated resource footprint (resource reservations).

When expanding a label, the router considers every neighboring site. A neighbor is skipped if it is occupied by another active virtual qubit, if the edge has no legal movement measurement, or if the required patch and edge reservations conflict with already scheduled work. When visiting a new node, the label is updated accordingly. For reliability-aware route search, a label is discarded as soon as its estimated accumulated risk exceeds τ_{safe} .

The implementation supports two queue orderings:

- **shortest**: labels are ordered by hop count, then time. If there are multiple equivalent shortest

paths possible, one is chosen randomly.

- **reliability**: labels are ordered by estimated risk, then time, then hop count.

Dominance pruning keeps the number of labels manageable. Labels are grouped by current patch and Hadamard frame. A new label is discarded if an existing label of the same group is better in time, hop count, estimated error, and footprint length, with at least one strict improvement. Conversely, labels dominated by the new label are removed. When the queue reaches the target, the router returns the corresponding route.

4.8 Reliability map

The reliability map is implemented in `reliability.py`. For each patch q_i , the map stores the smoothed activity ρ_i , the previous smoothed activity, the smoothed usage u_i , and the current weight w_i . At every syndrome-update cycle, the hidden noise model returns a syndrome sample for every site. The sample contains the number of detection events, the number of stabilizers, the normalized activity, and the hidden detection probability used for evaluation.

All the weights are updated each cycle according to the previous description in chapter 3.

4.9 Noise model

The hidden noise model is implemented in `noise.py`. The hidden model determines the true error probabilities used to score a run and to sample syndrome observations, while the reliability map only receives noisy observations and must infer routing weights from them.

For each patch, the hidden model samples a random phase at initialization. This phase is used in a sinusoidal drift term, giving every patch a time-varying noise contribution. A patch error probability is computed from the maximum of its base error and the global base site error, plus drift, plus an optional active-usage boost, multiplied by any active hotspot multiplier. Detection probability is computed similarly, but with the detection base and measurement-error parameters. Syndrome samples are then drawn from a binomial distribution over the site's stabilizer count.

Edges have two cases. A rhombic edge combines the configured edge base error with the average hidden site error of its endpoints, scaled by the edge risk factor. An inter-core edge instead uses the configured Bell-pair fidelity: its error probability is $1 - F_{\text{Bell}}$, again multiplied by active hotspot factors. Joint-measurement error is the joint-measurement base error plus the hidden edge error. Magic-state error is the magic base error plus the hidden site error. Wait error is the wait base error plus a scaled site error.

Hotspots are specified directly in the experiment file as resources, time intervals, and multipliers. A hotspot can target a site id or an edge id such as $u|v$. During its active interval, the corresponding hidden error and detection probabilities are multiplied by the hotspot factor. This provides a controllable way to evaluate whether reliability-aware routing avoids temporary noisy regions.

4.10 Strategies

Strategies are defined by `StrategyProfile` in `templates.py`. A profile specifies the route mode, whether reliability constraints are used, whether adaptive rerouting is enabled, and whether the Lovis-style compiler is used. The implemented strategies are summarized in Table 4.1.

Strategy	Route mode	Reliability	Partial rerouting
<code>shortest_path</code>	Shortest	No	No
<code>shortest_path_virtual</code>	Shortest	No	No
<code>static_reliability</code>	Reliability	Yes	No
<code>dynamic_reliability</code>	Reliability	Yes	Yes
<code>static_reliability_virtual</code>	Reliability	Yes	No
<code>dynamic_reliability_virtual</code>	Reliability	Yes	Yes

Table 4.1: Implemented strategy profiles.

The shortest-path strategies ignore reliability thresholds by passing $\tau_{\text{safe}} = 1$ to the router and selecting templates by hop count. This strategy provides the baseline for measuring reliability improvements.

Static reliability strategies use the current reliability map when a gate is planned. Route labels whose accumulated estimated risk exceeds τ_{safe} are pruned, and candidate templates are chosen using the weighted reliability-aware cost. Once an execution plan has been dispatched, the execution is never stopped and rerouted.

Dynamic reliability strategies add adaptive cancellation of risky future plan suffixes. If a planned but not-yet-started suffix becomes unsafe under the updated reliability map, the plan is cancelled and the corresponding logical gate becomes schedulable again. The replacement route is then generated by the same reliability-aware template mechanism used by the static strategy.

The reliability profiles also contain a bounded fallback mechanism. If no safe non-wait plan is available, the runtime can defer the gate for up to `no_wait_fallback_cycles`. After that limit, it generates unpruned candidates with $\tau_{\text{safe}} = 1$ and dispatches the lowest estimated risk route. This prevents an experiment from stalling indefinitely when the configured safety threshold is too strict for the current layout and noise state.

Algorithm 2 Label-setting routing algorithm

Require: source s , targets T , start cycle t_0 , frame f_0 , reliability map W

Require: reservation table $\mathcal{R}$, safety threshold τ_{safe} , mode m

Ensure: feasible route or no_feasible_route

```
1: if  $s \in T$  then
2:   return zero-hop route at  $s$ 
3: end if
4:  $\ell_0 \leftarrow (s, t_0, 0, 0, 0, f_0, (s), \emptyset, \emptyset)$  ▷ site, time, hops, estimated risk, true risk, frame, path,
   measurements, footprint
5:  $Q \leftarrow \{\ell_0\}, \quad L[(s, f_0)] \leftarrow \{\ell_0\}$ 
6:  $t_{\text{max}} \leftarrow t_0 + \text{max\_extra\_time}$ 
7: while  $Q$  is not empty do
8:    $\ell \leftarrow \text{POPBEST}(Q, m)$ 
9:   if  $\ell.q \in T$  then
10:    return route encoded by  $\ell$ 
11:   end if
12:   if  $\ell.t > t_{\text{max}}$  then
13:     continue
14:   end if
15:   for all feasible steps  $e$  from  $\ell.q$  do
16:     if  $e$  touches an occupied non-target patch then
17:       continue
18:     end if
19:     if  $e$  has no legal movement or teleportation measurement then
20:       continue
21:     end if
22:     if  $e$  conflicts with  $\mathcal{R}$  then
23:       continue
24:     end if
25:      $\hat{p}' \leftarrow 1 - (1 - \ell.\hat{p})(1 - \text{ESTIMATEDSTEP RISK}(e, W))$ 
26:      $p'_{\text{true}} \leftarrow 1 - (1 - \ell.p_{\text{true}})(1 - \text{HIDDENSTEP RISK}(e))$ 
27:     if  $m \neq \text{SHORTEST}$  and  $\hat{p}' > \tau_{\text{safe}}$  then
28:       continue
29:     end if
30:      $\ell' \leftarrow$  label obtained by appending  $e$  to  $\ell$ 
31:     update  $\ell'$  with new time, hop count, frame, path, risks, measurements, and footprint
32:     if  $\ell'$  is not dominated by labels in  $L[(\ell'.q, \ell'.f)]$  then
33:       remove labels dominated by  $\ell'$  from  $L[(\ell'.q, \ell'.f)]$ 
34:       add  $\ell'$  to  $L[(\ell'.q, \ell'.f)]$ 
35:       push  $\ell'$  into  $Q$ 
36:     end if
37:   end for
38: end while
39: return no_feasible_route
```

Chapter 5

Evaluation

In this chapter, we describe two simple experiments simulated using the program described in chapter 4, and analyze the obtained results.

5.1 Experiment 1: single CNOT in a single core

This experiment contains a single CNOT on one 3×3 core. A hotspot is placed on the central-left ancilla `c00_p10`, which is exactly the shared ancilla used by the shortest direct implementation between the fixed homes of q_0 and q_3 . This experiment isolates the core behavior expected from the reliability architecture: shortest-path routing should choose the geometrically direct operation, while reliability-aware routing should spend extra movement cycles to avoid the noisy patch.

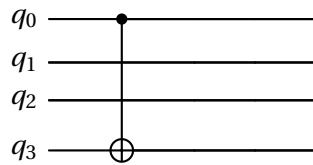


Figure 5.1: Input circuit for experiment 1.

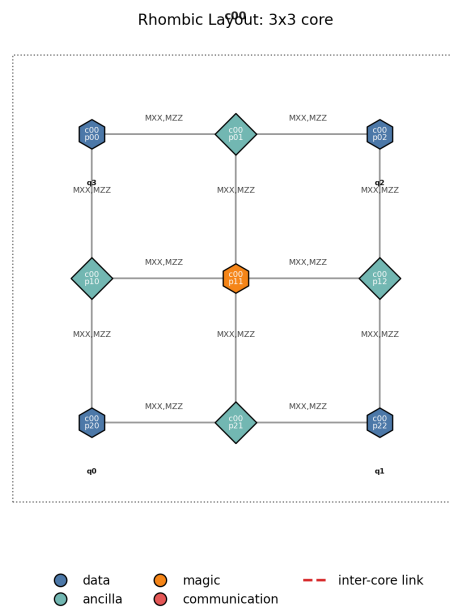


Figure 5.2: Input layout for experiment 1

Group	Parameter	Value
Experiment	Seed	4242
Experiment	Repetitions	1
Experiment	Layout size	9 sites, 12 edges
Experiment	Logical qubits	4
Experiment	Circuit operations	1
Timing	syndrome_period	1
Timing	frame_cycles	0
Timing	smooth_move_cycles	2
Timing	inter_core_cycles	5
Timing	joint_measure_cycles	3
Timing	magic_cycles	6
Timing	wait_cycles	1
Timing	cooldown_cycles	0
Timing	max_cycles	2000
Timing	scheduling_epoch	1
Algorithm	alpha	0.35
Algorithm	gamma	0.15
Algorithm	trend_coefficient	0.25
Algorithm	usage_heating_coefficient	0.20
Algorithm	tau_reroute_init	0.10
Algorithm	tau_min	0.025
Algorithm	tau_max	0.35
Algorithm	lambda_safe	0.70
Algorithm	beta	0.35
Algorithm	gain_target	0.015
Algorithm	local_reroute_horizon	24
Algorithm	max_local_wait	24
Algorithm	wait	false
Algorithm	no_wait_fallback_cycles	10
Algorithm	lookahead_gates	12
Algorithm	cp_sat_workers	1
Algorithm	risk_weight	125
Algorithm	space_time_weight	0.08
Algorithm	congestion_weight	4
Algorithm	magic_weight	4
Algorithm	lookahead_weight	0.8

Table 5.1: Input algorithm and timing parameters for the experiment 1.

Parameter	Value	Meaning
site_base_error	0.001	Baseline hidden site error.
edge_base_error	0.002	Baseline hidden edge error.
joint_error	0.004	Hidden joint-measurement operation error.
magic_error	0.008	Hidden magic-state import error.
frame_error	0	Hidden frame-update error.
wait_error	0.0004	Hidden waiting-memory error.
detection_base	0.012	Baseline syndrome detection probability.
detection_measurement_error	0.002	Syndrome measurement noise.
usage_boost	0.012	Extra syndrome activity when a patch is used.
drift_amplitude	0.004	Periodic drift amplitude.
drift_period	47	Periodic drift period in cycles.
Hotspot	c00_p10, cycles 0–1200, multiplier 50	Central-left ancilla is much noisier during the full run.

Table 5.2: Noise model and hotspot parameters for the experiment 1.

5.1.1 Route choices

Shortest path - step 1, cycles 0-6, repetition 0, seed 4242, gate 0: CNOT q0, q3 (direct)
direct: c00_p10|c00_p00 (MXX); c00_p20|c00_p10 (MZZ)

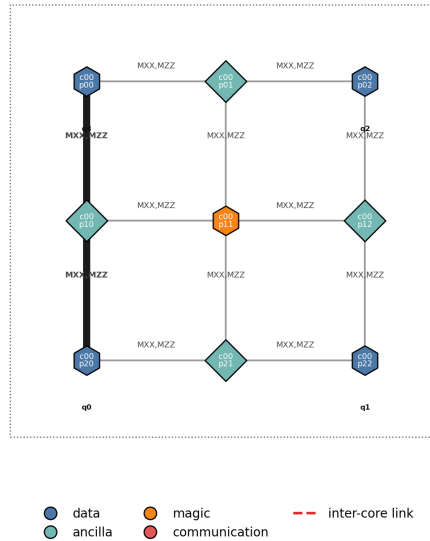


Figure 5.3: Route choice for experiment 1 with `shortest_path` strategy.

shortest_path. The fixed homes are $q_0 \mapsto \text{c00_p20}$ and $q_3 \mapsto \text{c00_p00}$. The shortest direct CNOT uses c00_p10 as the shared ancilla with measurements MXX and MZZ. Because this strategy is reliability-blind for route choice, it does not avoid the hotspot and finishes in 6 cycles.

Shortest Path Lovis - step 1, cycles 0-6, repetition 0, seed 4242, gate 0: CNOT q0, q3 (direct)
direct: c00_p10|c00_p00 (MXX); c00_p20|c00_p10 (MZZ)

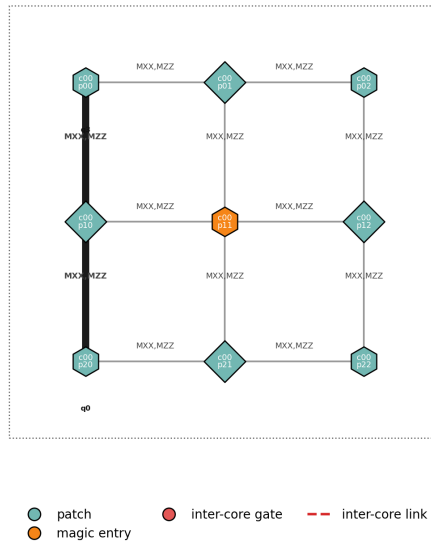


Figure 5.4: Route choice for experiment 1 with `shortest_path_virtual` strategy.

`shortest_path_virtual`. This strategy uses virtual mapping, but shortest-path virtual does not score placements using reliability. For this seed and tie breaking, it maps the two virtual qubits to the same direct CNOT pattern as the fixed shortest strategy. It therefore also crosses the hotspot through c00_p10.

[illegible]

The diagram illustrates a quantum circuit architecture with eight qubits arranged in a square grid:

- Qubits:** Labeled from $c00-q00$ to $c03-q03$.
- Connections:**
 - Thick black lines represent **inter-core links** between $c00-q01$, $c01-q01$, $c02-q01$, and $c03-q01$.
 - A pink circle highlights the **movement destination** of qubit $c01-q01$.
 - Thin grey lines represent other connections between adjacent qubits.
- Legend:**
 - Blue circle: data
 - Orange circle: magic
 - Teal circle: ancilla
 - Red circle: communication
 - Red dashed line: inter-core link
 - Pink circle: movement destination

Static Reliability Lovis - step 1, cycles 0-6, repetition 0, seed 4242, gate 0: CNOT q0, q3 (direct)
direct: c00_p11|c00_p01 (MZZ); c00_p01|c00_p00 (MXX)

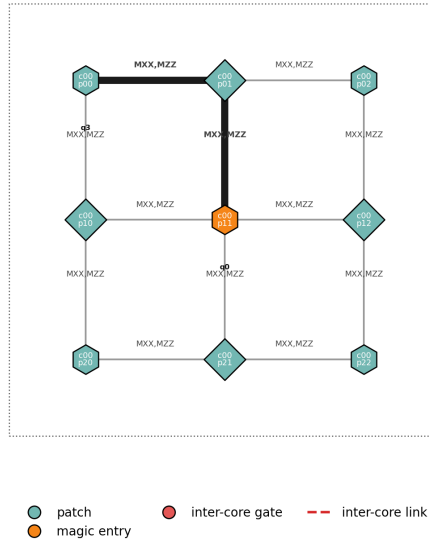
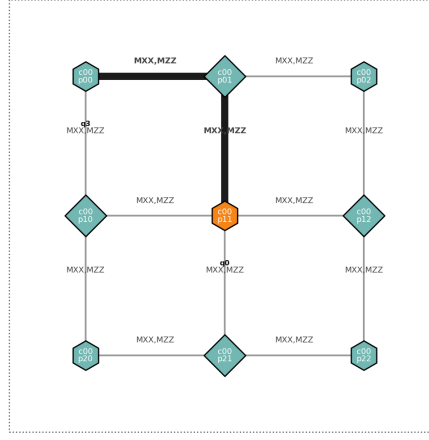


Figure 5.7: Route choice for experiment 1 with `static_reliability_virtual` strategy.

`static_reliability_virtual`. Here the reliability-aware virtual mapper can choose the physical homes before issuing the gate. It maps the operands directly to a legal CNOT triple that avoids c00_p10: $q_0 \mapsto c00_p11$ and $q_3 \mapsto c00_p00$, with c00_p01 used as the shared ancilla. This keeps the direct 6-cycle latency while avoiding the hotspot.

Dynamic Reliability Lovis - step 1, cycles 0-6, repetition 0, seed 4242, gate 0: CNOT q0, q3 (direct)
direct: c00_p11|c00_p01 (MZZ); c00_p01|c00_p00 (MXX)



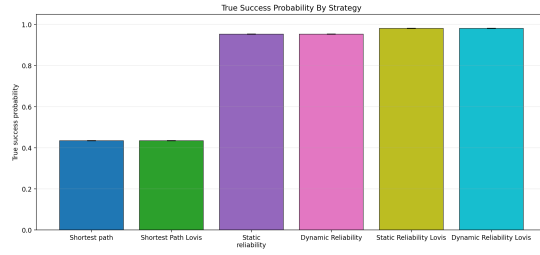
● patch
● magic entry
● inter-core gate
--- inter-core link

Figure 5.8: Route choice for experiment 1 with dynamic_reliability_virtual strategy.

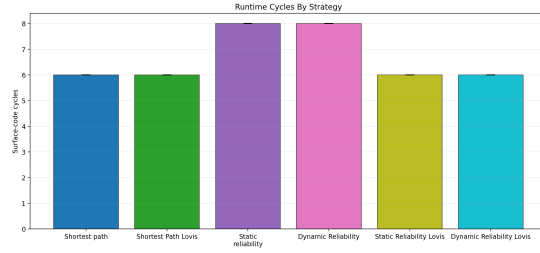
dynamic_reliability_virtual. The dynamic virtual strategy selects the same safe direct placement as static reliability Lovis. As in the fixed dynamic run, no adaptive reroute occurs because the single gate completes immediately after it is issued.

Strategy	True success probability	Estimated success probability	Total cycles
shortest_path	0.434973	0.737500	6
shortest_path_virtual	0.434973	0.737500	6
static_reliability	0.952886	1.000000	8
dynamic_reliability	0.952886	1.000000	8
static_reliability_virtual	0.981648	1.000000	6
dynamic_reliability_virtual	0.981648	1.000000	6

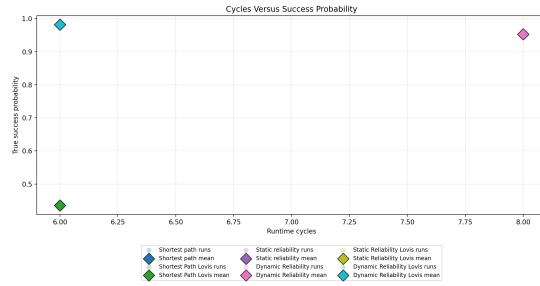
Table 5.3: Experiment 1 result summary.



(a) True success.



(b) Cycle latency.



(c) Latency-success tradeoff.

Figure 5.9: Graph for the experiment 1.

The results match the expected behavior. The shortest strategies are fast, but they use the hot shared ancilla and therefore have the worst true success probability. Fixed-data reliability improves the true success probability from 0.435 to 0.953 by routing around the hotspot, but this costs two extra cycles. The Lovis reliability strategies perform best because virtual-qubit mapping avoids the hotspot before routing is needed: they preserve the 6-cycle latency of the direct implementation while increasing the true success probability to 0.982. This experiment therefore confirms that the reliability map affects template selection and that reliability-aware virtual placement can remove routing overhead when a safe direct placement exists.

5.2 Experiment 2 : distant CNOTs in distributed cores

This experiment contains four 3×3 cores and three distant CNOTs sharing q_0 :

$$\text{CNOT}(q_0, q_{15}), \quad \text{CNOT}(q_0, q_5), \quad \text{CNOT}(q_0, q_{10}).$$

The entire c00 core is configured as a hotspot. This experiment tests whether reliability-aware routing can avoid a noisy core, and whether virtual-qubit mapping can place operands directly in a safe core instead of inheriting the fixed input mapping.

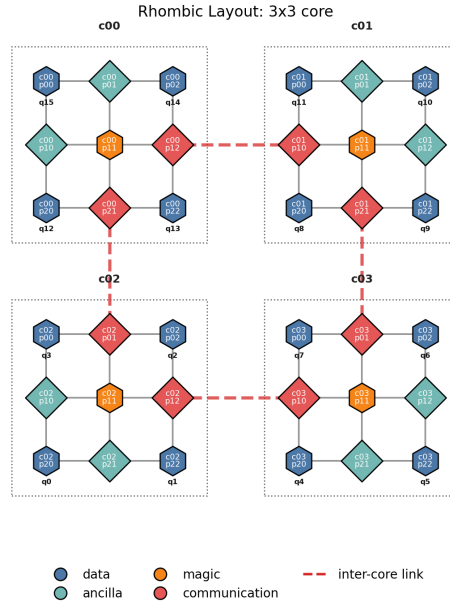


Figure 5.10: Input layout for the experiment 2.

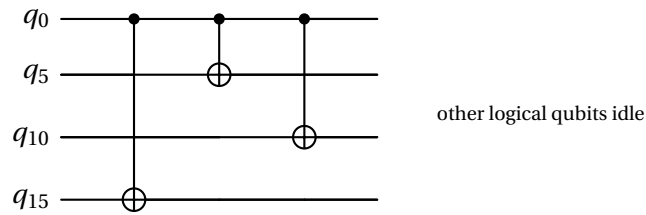


Figure 5.11: Compact drawing of the input circuit for experiment 2. Only the active wires are drawn.

Group	Parameter	Value
Experiment	Seed	6969
Experiment	Repetitions	1
Experiment	Layout size	36 sites, 52 edges
Experiment	Logical qubits	16
Experiment	Circuit operations	3
Timing	syndrome_period	1
Timing	frame_cycles	0
Timing	smooth_move_cycles	2
Timing	inter_core_cycles	5
Timing	joint_measure_cycles	3
Timing	magic_cycles	6
Timing	wait_cycles	1
Timing	cooldown_cycles	0
Timing	max_cycles	2000
Timing	scheduling_epoch	1
Algorithm	alpha	0.35
Algorithm	gamma	0.15
Algorithm	trend_coefficient	0.25
Algorithm	usage_heating_coefficient	0.20
Algorithm	tau_reroute_init	0.10
Algorithm	tau_min	0.025
Algorithm	tau_max	0.35
Algorithm	lambda_safe	0.70
Algorithm	beta	0.35
Algorithm	gain_target	0.015
Algorithm	local_reroute_horizon	24
Algorithm	max_local_wait	24
Algorithm	wait	true
Algorithm	no_wait_fallback_cycles	10
Algorithm	lookahead_gates	12
Algorithm	cp_sat_workers	1
Algorithm	risk_weight	125
Algorithm	space_time_weight	0.08
Algorithm	congestion_weight	4
Algorithm	magic_weight	4
Algorithm	lookahead_weight	0.8

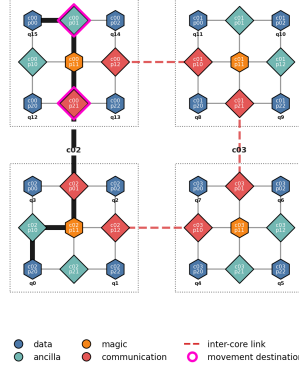
Table 5.4: Input algorithm and timing parameters for experiment 2.

Parameter	Value	Meaning
site_base_error	0.001	Baseline hidden site error.
edge_base_error	0.002	Baseline hidden edge error.
joint_error	0.004	Hidden joint-measurement operation error.
magic_error	0.008	Hidden magic-state import error.
frame_error	0	Hidden frame-update error.
wait_error	0.0004	Hidden waiting-memory error.
detection_base	0.012	Baseline syndrome detection probability.
detection_measurement_error	0.002	Syndrome measurement noise.
usage_boost	0.012	Extra syndrome activity when a patch is used.
drift_amplitude	0.004	Periodic drift amplitude.
drift_period	47	Periodic drift period in cycles.
Hotspot	c00_p00, c00_p01, c00_p02, c00_p10, c00_p11, c00_p12, c00_p20, c00_p21, c00_p22; cycles 0–1200; multiplier 15	The complete c00 core is noisy during the full run.

Table 5.5: Noise model and hotspot parameters for the experiment 2.

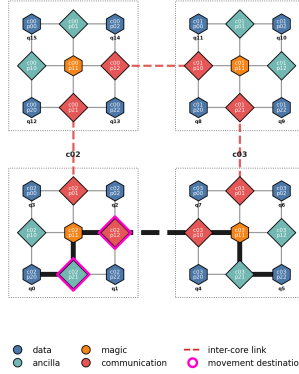
5.2.1 Route choices

Shortest path - step 1, cycles 0-24, repetition 0, seed 6969, gate 0: CNOT q_0, q_{15} (move_both)
 $q_0: c02_p20 \rightarrow c02_p10 \rightarrow c00_p21; q_{15}: c00_p00 \rightarrow c00_p01$; joint: $c02_p10|c02_p11$ (MZZ); $c02_p11|c02_p01$ (MXX); $c00_p21|c00_p11$ (MZZ); $c00_p11|c00_p01$ (MXX)



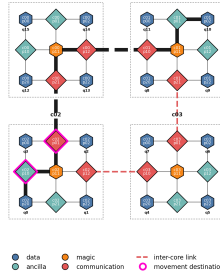
(a) CNOT(q_0, q_{15})

Shortest path - step 2, cycles 24-48, repetition 0, seed 6969, gate 1: CNOT q_0, q_5 (move_both)
 $q_0: c02_p20 \rightarrow c02_p21; q_5: c03_p22 \rightarrow c03_p21 \rightarrow c02_p12$; joint: $c03_p21|c03_p11$ (MZZ); $c03_p11|c03_p10$ (MXX); $c02_p11|c02_p12$ (MXX); $c02_p11|c02_p11$ (MZZ)



(b) CNOT(q_0, q_5)

Shortest path - step 3, cycles 48-88, repetition 0, seed 6969, gate 2: CNOT q_0, q_{10} (move_both)
 $q_0: c02_p20 \rightarrow c02_p10; q_{10}: c01_p02 \rightarrow c01_p01 \rightarrow c00_p12 \rightarrow c02_p01$; joint: $c01_p01|c01_p11$ (MZZ); $c01_p11|c01_p10$ (MXX); $c00_p12|c00_p11$ (MZZ); $c00_p11|c00_p21$ (MXX); $c02_p10|c02_p11$ (MZZ); $c02_p11|c02_p01$ (MXX)



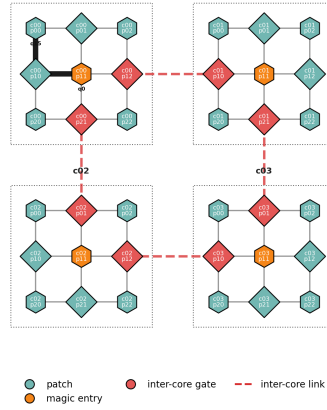
(c) CNOT(q_0, q_{10})

Figure 5.12: Route choices for shortest_path on the three distant CNOTs.

shortest_path. The fixed input mapping places q_0 in core c02, while q_{15} , q_5 , and q_{10} are in other cores. Because shortest path ignores reliability for route choice, all three gates use move_both

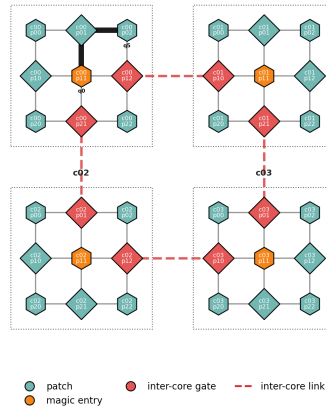
templates chosen by hop count. The first and third gates route through the noisy c00 core because those paths are geometrically short under the fixed mapping. This gives the lowest latency among fixed strategies, 88 cycles, but the lowest true success probability.

Shortest Path Lovis - step 1, cycles 0-6, repetition 0, seed 6969, gate 0: CNOT q_0, q_{15} (direct)
direct: $c00_p10|c00_p00$ (MXX); $c00_p11|c00_p10$ (MZZ)



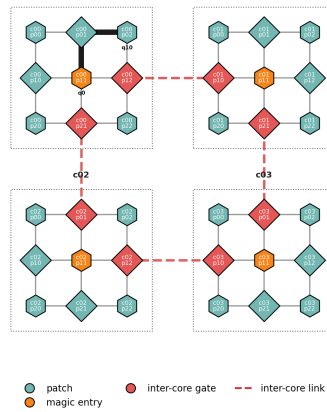
(a) CNOT(q_0, q_{15})

Shortest Path Lovis - step 2, cycles 6-12, repetition 0, seed 6969, gate 1: CNOT q_0, q_5 (direct)
direct: $c00_p01|c00_p02$ (MXX); $c00_p11|c00_p01$ (MZZ)



(b) CNOT(q_0, q_5)

Shortest Path Lovis - step 3, cycles 12-18, repetition 0, seed 6969, gate 2: CNOT q_0, q_{10} (direct)
direct: $c00_p11|c00_p01$ (MZZ); $c00_p01|c00_p02$ (MXX)



(c) CNOT(q_0, q_{10})

Figure 5.13: Route choices for `shortest_path_virtual` on the three distant CNOTs.

shortest_path_virtual. The virtual mapper removes the fixed-home constraint, so each gate can be performed as a direct local CNOT. However, this shortest-path virtual strategy is still reliability-blind during placement. It maps the CNOTs into $c00$, which is the hotspot core, because that placement is geometrically valid and direct. It therefore obtains only 18 cycles, but its true success probability is limited by the noisy core.

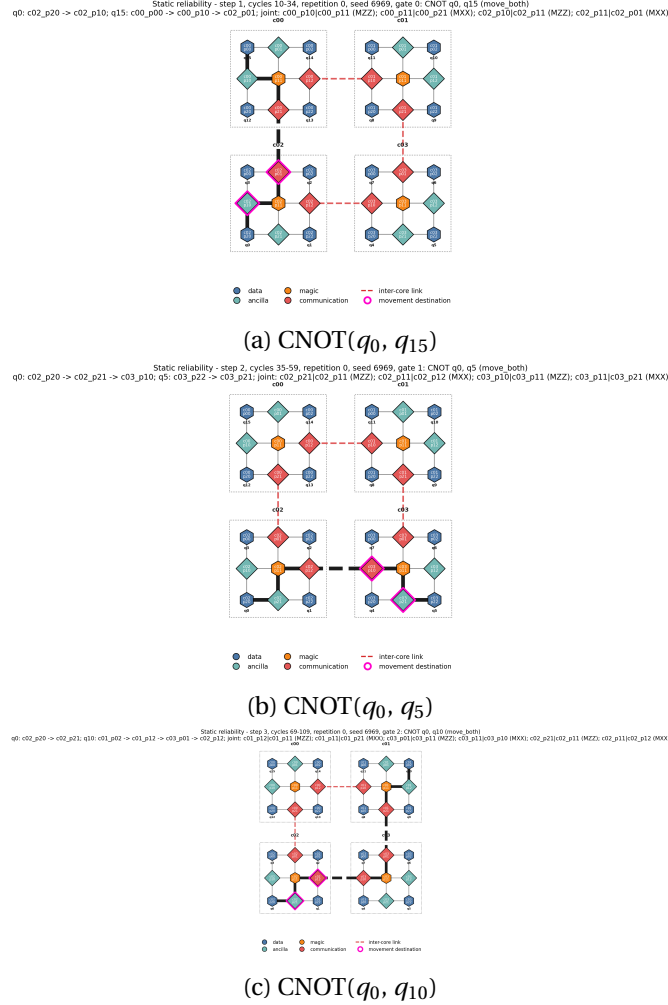


Figure 5.14: Route choices for `static_reliability` on the three distant CNOTs.

static_reliability. With fixed data homes, static reliability cannot freely remap operands away from their initial cores. It can only choose safer routes and templates. The first gate is delayed until cycle 10 because the safety threshold initially rejects the available plans because of the unavoidable hotspot in $c00$; after the bounded fallback it selects the lowest-risk available route. The selected routes still require inter-core teleportations, but they reduce the expected exposure to the hotspot

compared with shortest path. This improves true success probability from 0.322 to 0.452, at the cost of increasing the latency from 88 to 109 cycles.

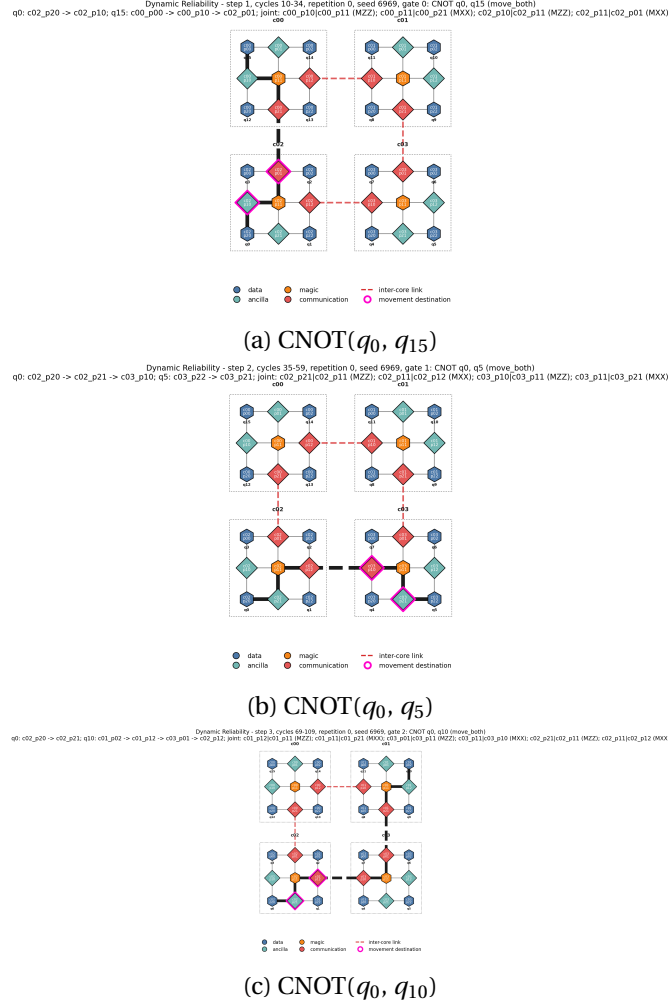
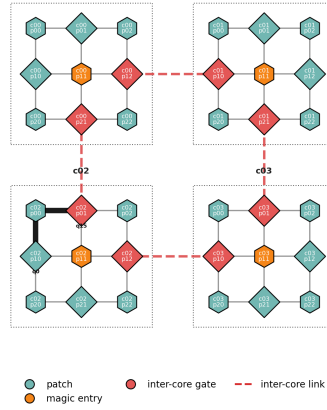


Figure 5.15: Route choices for dynamic_reliability on the three distant CNOTs.

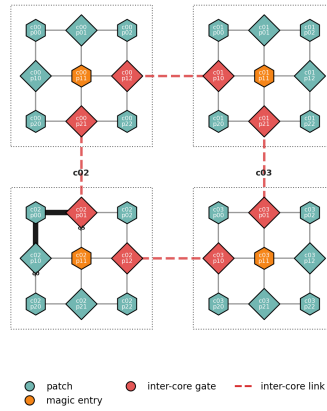
dynamic_reliability. The dynamic fixed strategy chooses the same routes as static reliability in this experiment. No successful partial reroute is recorded because the routes that are dispatched do not have future suffixes that cross the rerouting threshold before starting. The result is therefore identical to static reliability: better reliability than shortest path, but with the same waiting and routing overhead.

Static Reliability Lovis - step 1, cycles 0-6, repetition 0, seed 6969, gate 0: CNOT q_0, q_{15} (direct)
direct: c02_p10|c02_p00 (MZZ); c02_p00|c02_p01 (MXX)



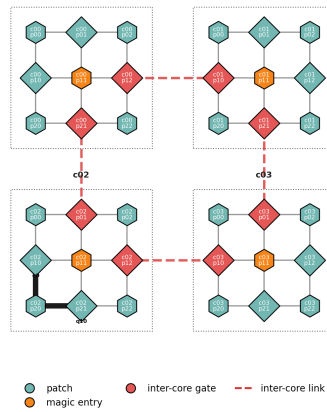
(a) CNOT(q_0, q_{15})

Static Reliability Lovis - step 2, cycles 6-12, repetition 0, seed 6969, gate 1: CNOT q_0, q_5 (direct)
direct: c02_p10|c02_p20 (MZZ); c02_p00|c02_p21 (MXX)



(b) CNOT(q_0, q_5)

Static Reliability Lovis - step 3, cycles 12-18, repetition 0, seed 6969, gate 2: CNOT q_0, q_{10} (direct)
direct: c02_p10|c02_p20 (MZZ); c02_p20|c02_p21 (MXX)

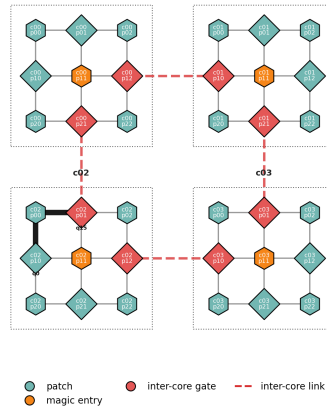


(c) CNOT(q_0, q_{10})

Figure 5.16: Route choices for static_reliability_virtual on the three distant CNOTs.

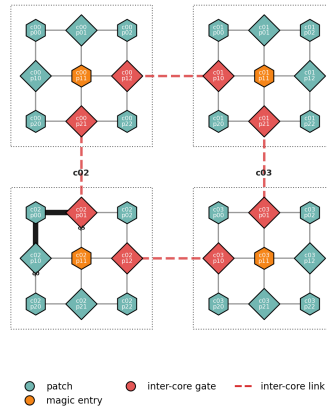
`static_reliability_virtual`. This strategy shows the intended interaction between reliability-aware mapping and Lovis virtual qubits. Instead of inheriting the fixed homes or placing the operands in the noisy `c00` core, the mapper places the active virtual qubits directly on safe legal CNOT triples in core `c02`. Each logical CNOT is then direct and requires no movement route. This gives the same 18-cycle latency as shortest-path Lovis, but the true success probability increases to 0.933.

Dynamic Reliability Lovis - step 1, cycles 0-6, repetition 0, seed 6969, gate 0: CNOT q_0, q_{15} (direct)
direct: $c02_p10|c02_p00$ (MZZ); $c02_p00|c02_p01$ (MXX)



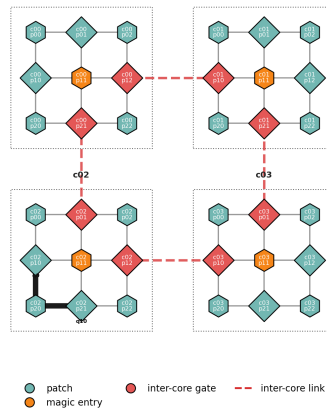
(a) $\text{CNOT}(q_0, q_{15})$

Dynamic Reliability Lovis - step 2, cycles 6-12, repetition 0, seed 6969, gate 1: CNOT q_0, q_5 (direct)
direct: $c02_p10|c02_p00$ (MZZ); $c02_p00|c02_p01$ (MXX)



(b) $\text{CNOT}(q_0, q_5)$

Dynamic Reliability Lovis - step 3, cycles 12-18, repetition 0, seed 6969, gate 2: CNOT q_0, q_{10} (direct)
direct: $c02_p10|c02_p20$ (MZZ); $c02_p20|c02_p21$ (MXX)



(c) $\text{CNOT}(q_0, q_{10})$

Figure 5.17: Route choices for dynamic_reliability_virtual on the three distant CNOTs.

dynamic_reliability_virtual. The dynamic Lovis run is identical to static reliability Lovis for this input. The reliability-aware virtual mapper already finds safe direct placements in c02, so there is no risky future route suffix to cancel.

Strategy	True success probability	Estimated success probability	Total cycles
shortest_path	0.321732	0.231175	88
shortest_path_virtual	0.501939	0.145697	18
static_reliability	0.452086	0.293098	109
dynamic_reliability	0.452086	0.293098	109
static_reliability_virtual	0.933010	0.966012	18
dynamic_reliability_virtual	0.933010	0.966012	18

Table 5.6: Experiment 2 result summary.

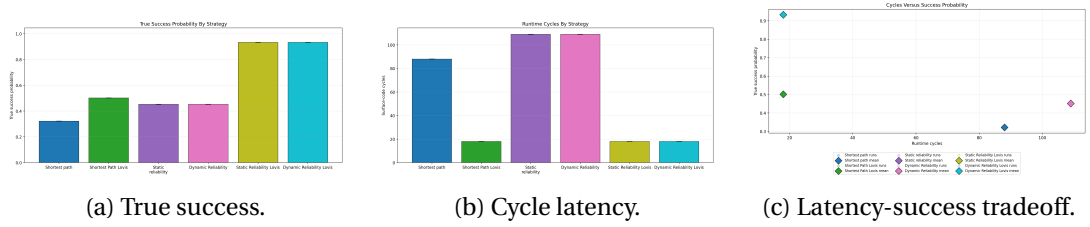


Figure 5.18: Graph for the experiment 2.

The distant-CNOT results show the two benefits and the main cost of the proposed architecture. First, reliability-aware fixed routing improves the true success probability relative to fixed shortest path because it uses the reliability map to reduce hotspot exposure. The price is latency: the reliability-aware fixed strategies wait and then execute longer movement templates, increasing the total runtime from 88 to 109 cycles. Second, reliability-aware virtual mapping is substantially stronger than routing alone when virtual placement is allowed. It places the active qubits directly in the safe c02 core, so the circuit avoids inter-core movement entirely and finishes in 18 cycles with a true success probability of 0.933. The shortest-path Lovis result is the important control case: virtual mapping alone is not sufficient, because reliability-blind placement can choose the noisy core. The improvement therefore comes specifically from combining the reliability map with virtual-qubit mapping.

Chapter 6

Future Work

The simulator and experiments presented in this report provide a first implementation of reliability-aware routing for distributed surface-code layouts. Several directions remain open to make the evaluation more realistic, to better understand the contribution of each design choice, and to scale the approach to larger circuits.

6.1 Circuit-Level Noise Modeling with STIM

The current simulator uses a hidden stochastic noise model to generate true operation errors and syndrome observations. This is useful for controlled experiments because hotspots, drifts, and usage effects can be isolated precisely. However, a more physically grounded evaluation should connect the runtime architecture to circuit-level noise simulation. A natural next step is to model the atomic operations with STIM [8], including reset, measurement, joint measurement, magic-state importation, and inter-core teleportation primitives. The reliability map could then be updated from syndromes produced by explicit stabilizer circuits instead of synthetic detection-event samples.

This would make the reported success probabilities closer to the behavior of an actual surface-code implementation. It would also allow the architecture to be tested against different decoder assumptions, different physical error channels, biased noise, measurement errors, leakage-inspired models, and correlated errors. In such a setting, the true success probability would no longer come from an abstract operation-level model, but from the sampled stabilizer checks of the generated noisy circuits.

6.2 Reliability-Architecture Ablation and Parameter Optimization

The reliability-aware strategies contain several interacting mechanisms: the syndrome smoothing parameter α , the usage smoothing parameter γ , the usage-bias correction, the trend term, the safety threshold τ_{safe} , the rerouting threshold τ_{reroute} , bounded waiting, fallback behavior, and dynamic partial rerouting. The current experiments demonstrate that the complete architecture can improve fidelity, but they do not yet quantify how much each individual mechanism contributes.

Future experiments should therefore perform systematic ablations. For example, one can disable the usage-bias correction, remove the trend term, vary the safety threshold, compare static and dynamic rerouting under longer circuits, and measure the resulting fidelity gain and latency overhead. This would show which parts of the architecture are essential and which parts mainly add complexity.

A related direction is to search for optimal parameter values. The threshold values and usage-bias coefficients could be swept over many layouts, circuits, and hotspot patterns to find regimes where reliability-aware routing gives the best success-probability improvement per added cycle. More ambitiously, one could try to prove that the dynamic threshold algorithm is guaranteed to converge towards the optimal value.

6.3 Stricter Rhombic Core Layout

The simulation could also be made stricter with respect to the rhombic layout described by Lovis [16]. For example, rhombic-shaped cores could be enforced, and inferred from their four size parameters, and reject incompatible layouts.

With enforced rhombic core geometry, we could also derive legal joint measurements entirely from patch orientations, relative boundaries, and Hadamard frame, instead of relying on legal measurements stored in the edge description.

6.4 Scaling to Larger Circuits and Architectures

The experiments in this report are intentionally small, which makes route choices easy to inspect and explain. A more extensive evaluation should test larger cores, more logical qubits, more distributed cores, and deeper circuits. The benchmark set should include random CNOT layers, structured circuits with repeated interactions, magic-heavy circuits, and circuits whose interaction graph changes over time. These experiments would show how reliability-aware architectures' fidelity gain and latency scale when many gates compete for the same routing resources.

To improve scalability, reducing the algorithms complexities would have large benefits. Reliability-aware template generation can become expensive because many candidate placements and routes may need to be evaluated before a gate is dispatched.

Chapter 7

Conclusion

This project studied whether quantum circuit execution fidelity can be improved by leveraging syndrome extraction for runtime reliability-aware routing and mapping decisions in distributed surface-code architectures. The central idea was to move beyond purely geometric shortest-path routing and let the controller observe syndrome activity, build a reliability map over logical patches, and use this map to avoid temporarily noisy regions. To evaluate this idea, we implemented a discrete-event simulator for distributed surface-code layouts, including time-slice scheduling, fixed and virtual logical-qubit strategies, resource reservations, atomic lattice-surgery operations, inter-core teleportation, reliability-aware routing, and adaptive rerouting.

The main result is that reliability-aware routing can significantly improve the simulated success probability when the shortest route crosses a noisy region. In the first experiment, shortest-path routing selected the direct CNOT through the hotspot and obtained a true success probability of 0.435. Fixed-data reliability-aware routing avoided the hotspot and increased the true success probability to 0.953, at the cost of increasing the latency from 6 to 8 cycles. When reliability-aware routing was combined with virtual-qubit mapping, the simulator could place the operands directly on a safe CNOT pattern, reaching a true success probability of 0.982 without increasing the latency.

The second experiment showed the same principle at a larger scale. Fixed shortest-path routing completed the circuit in 88 cycles, but repeatedly used routes exposed to the noisy core and obtained a true success probability of 0.322. Fixed reliability-aware routing improved this to 0.452 by choosing safer routes, but required 109 cycles because it sometimes had to wait and use longer routes. The strongest result came from reliability-aware virtual-qubit mapping: by placing the active virtual qubits directly in a safe core, the simulator completed the three CNOTs in 18 cycles with a true success probability of 0.933. The comparison with shortest-path virtual-qubit is important: virtual mapping alone was not sufficient, because reliability-blind placement could still choose the noisy core. The improvement came from combining virtual placement with the reliability map.

These results support the main thesis of the project. Reliability information from syndrome measurements can be lifted from the error-correction layer into the routing and scheduling layer, where it can guide logical state movement and logical-qubit placement. This advances the status quo over shortest-path routing because it treats the architecture as a time-varying reliability landscape rather than a static unweighted graph. In particular, the controller can distinguish between a short but noisy route and a longer but safer route, and it can exploit virtual-qubit mapping to avoid unnecessary movement altogether when a safe direct placement exists.

The work also clarifies the tradeoff introduced by reliability awareness. In fixed-data mode, avoiding a hotspot may require additional teleportations or waiting, so the success-probability improvement can come with cycle-latency overhead. In virtual-qubit mode, however, the mapper can sometimes convert a routing problem into a placement problem. When a safe direct placement exists, reliability-aware virtual mapping can improve fidelity without adding route length. This suggests that reliability-aware routing and virtual-qubit compilation are complementary: routing handles unavoidable movement, while virtual mapping reduces the need for movement in the first place.

In summary, this project shows that quantum circuit execution fidelity can be improved by leveraging syndrome extraction for runtime reliability-aware routing and mapping decisions in distributed surface-code architectures. This improvement comes at the cost of additional latency.

Bibliography

- [1] Saman Ahmadi, Andrea Raith, and Mahdi Jalili. “A Fast and Simple Algorithm for the Resource Constrained Shortest Path Problem”. In: *33rd Annual European Symposium on Algorithms (ESA 2025)*. Vol. 351. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum fuer Informatik, 2025, 97:1–97:15. DOI: 10.4230/LIPIcs.ESA.2025.97. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2025.97>.
- [2] Avimita Chatterjee, Subrata Das, and Swaroop Ghosh. “Lattice Surgery for Dummies”. In: *Sensors* 25.6 (2025), p. 1854. DOI: 10.3390/s25061854. URL: <https://doi.org/10.3390/s25061854>.
- [3] Andrew N. Cleland. “An Introduction to the Surface Code”. In: *SciPost Physics Lecture Notes* (2022), p. 49. DOI: 10.21468/SciPostPhysLectNotes.49. URL: <https://doi.org/10.21468/SciPostPhysLectNotes.49>.
- [4] Marc Grau Davis, Joaquin Chung, Dirk Englund, and Rajkumar Kettimuthu. *Towards Distributed Quantum Computing by Qubit and Gate Graph Partitioning Techniques*. 2023. DOI: 10.48550/arXiv.2310.03942. arXiv: 2310.03942 [quant-ph]. URL: <https://arxiv.org/abs/2310.03942>.
- [5] Yongshan Ding, Adam Holmes, Ali Javadi-Abhari, Diana Franklin, Margaret Martonosi, and Frederic T. Chong. “Magic-State Functional Units: Mapping and Scheduling Multi-Level Distillation Circuits for Fault-Tolerant Quantum Architectures”. In: *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture*. MICRO 2018. IEEE Computer Society, 2018, pp. 828–840. DOI: 10.1109/MICRO.2018.00072. URL: <https://doi.org/10.1109/MICRO.2018.00072>.
- [6] Bálint Domokos, Áron Márton, and János K. Asbóth. “Characterization of Errors in a CNOT between Surface Code Patches”. In: *Quantum* 8 (2024), p. 1577. DOI: 10.22331/q-2024-12-27-1577. URL: <https://doi.org/10.22331/q-2024-12-27-1577>.
- [7] Zefan Du, Shuwen Kan, Samuel A. Stein, Zhiding Liang, Ang Li, and Ying Mao. *Hardware-Aware Compilation for Chip-to-Chip Coupler-Connected Modular Quantum Systems*. 2025. DOI: 10.48550/arXiv.2505.09036. arXiv: 2505.09036 [quant-ph]. URL: <https://arxiv.org/abs/2505.09036>.

- [8] Craig Gidney. “Stim: A Fast Stabilizer Circuit Simulator”. In: *Quantum* 5 (2021), p. 497. DOI: 10.22331/q-2021-07-06-497. URL: <https://doi.org/10.22331/q-2021-07-06-497>.
- [9] Google Quantum AI and Collaborators. “Quantum Error Correction below the Surface Code Threshold”. In: *Nature* 638 (2025), pp. 920–926. DOI: 10.1038/s41586-024-08449-y. URL: <https://doi.org/10.1038/s41586-024-08449-y>.
- [10] Kou Hamada, Yasunari Suzuki, and Yuuki Tokunaga. “Efficient and High-Performance Routing of Lattice-Surgery Paths on Three-Dimensional Lattice”. In: *Quantum* 10 (2026), p. 2061. DOI: 10.22331/q-2026-04-13-2061. URL: <https://doi.org/10.22331/q-2026-04-13-2061>.
- [11] Laura S. Herzog, Lucas Berent, Aleksander Kubica, and Robert Wille. *Exploiting Movable Logical Qubits for Lattice Surgery Compilation*. 2025. DOI: 10.48550/arXiv.2512.04169. arXiv: 2512.04169 [quant-ph]. URL: <https://arxiv.org/abs/2512.04169>.
- [12] Dominic Horsman, Austin G. Fowler, Simon Devitt, and Rodney Van Meter. “Surface Code Quantum Computing by Lattice Surgery”. In: *New Journal of Physics* 14 (2012), p. 123011. DOI: 10.1088/1367-2630/14/12/123011. URL: <https://doi.org/10.1088/1367-2630/14/12/123011>.
- [13] Yifei Huang, Pascal Jahan Elahi, Kan He, Jinchuan Hou, and Shusen Liu. *TRAM: A Transverse Relaxation Time-Aware Qubit Mapping Algorithm for NISQ Devices*. 2025. DOI: 10.48550/arXiv.2511.16051. arXiv: 2511.16051 [quant-ph]. URL: <https://arxiv.org/abs/2511.16051>.
- [14] Cody Jones. “Multilevel Distillation of Magic States for Quantum Computing”. In: *Physical Review A* 87.4 (2013), p. 042305. DOI: 10.1103/PhysRevA.87.042305. URL: <https://doi.org/10.1103/PhysRevA.87.042305>.
- [15] Batuhan Keskin, Cameron Afradi, Sylvain Lovis, Maurizio Palesi, Pau Escofet, Carmen G. Almudever, and Edoardo Charbon. *Lattice Surgery Aware Resource Analysis for the Mapping and Scheduling of Quantum Circuits for Scalable Modular Architectures*. 2025. DOI: 10.48550/arXiv.2511.21885. arXiv: 2511.21885 [quant-ph]. URL: <https://arxiv.org/abs/2511.21885>.
- [16] Sylvain Lovis. *Mapping and Scheduling Quantum Circuits on Distributed Cores with Logical Qubits*. Semester project. EPFL, 2025.
- [17] Abtin Molavi, Amanda Xu, Swamit Tannu, and Aws Albarghouthi. *Dependency-Aware Compilation for Surface Code Quantum Architectures*. 2024. DOI: 10.48550/arXiv.2311.18042. arXiv: 2311.18042 [quant-ph]. URL: <https://arxiv.org/abs/2311.18042>.
- [18] Prakash Murali, Jonathan M. Baker, Ali Javadi-Abhari, Frederic T. Chong, and Margaret Martonosi. “Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers”. In: *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS 2019. Association

- for Computing Machinery, 2019, pp. 1015–1029. DOI: 10.1145/3297858.3304075. URL: <https://doi.org/10.1145/3297858.3304075>.
- [19] Joshua Ramette, Josiah Sinclair, Nikolas P. Breuckmann, and Vladan Vuletić. “Fault-Tolerant Connection of Error-Corrected Qubits with Noisy Links”. In: *npj Quantum Information* 10 (2024), p. 58. DOI: 10.1038/s41534-024-00855-4. URL: <https://doi.org/10.1038/s41534-024-00855-4>.
- [20] C. Ryan-Anderson, N. C. Brown, C. H. Baldwin, J. M. Dreiling, C. Foltz, J. P. Gaebler, T. M. Gatterman, N. Hewitt, C. Holliman, C. V. Horst, J. Johansen, D. Lucchetti, T. Mengle, M. Matheny, Y. Matsuoka, K. Mayer, M. Mills, S. A. Moses, B. Neyenhuis, J. Pino, P. Siegfried, R. P. Stutz, J. Walker, and D. Hayes. *High-Fidelity and Fault-Tolerant Teleportation of a Logical Qubit Using Transversal Gates and Lattice Surgery on a Trapped-Ion Quantum Computer*. 2024. DOI: 10.48550/arXiv.2404.16728. arXiv: 2404.16728 [quant-ph]. URL: <https://arxiv.org/abs/2404.16728>.